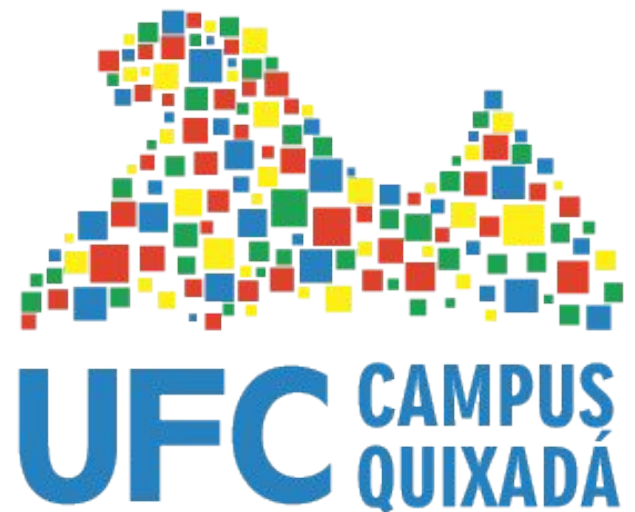


Arquitetura de Software

Sidarta Carvalho



Padrões e Estilos Arquiteturais

Padrões de arquitetura

- Padrões arquiteturais são meios de capturar estruturas e decisões de projeto que se provaram boas... isso indica que podem ser reutilizadas!
- Um padrão arquitetural é um conjunto de decisões que foram encontradas, de forma repetida, na prática de projeto de sistemas.

- **Podemos inventar padrões?**

Padrões de arquitetura

- Por definição, padrões são encontrados a partir de práticas, logo, não são inventados, mas descobertos!
- Padrões geralmente emergem em reação as mudanças no ambiente
 - Conforme as circunstâncias mudam, novos padrões vão sendo descobertos

Padrões de arquitetura

- O projeto de uma arquitetura geralmente parte da junção de diversos outros padrões
 - Seleção de padrões
 - Adaptação de padrões
 - Combinação de padrões
- O arquiteto de software deve decidir:
 - Como encaixar o padrão escolhido no projeto da arquitetura do sistema proposto?
 - Quais as restrições do sistema proposto?

Padrões de arquitetura

- Estilos arquiteturais
 - Geralmente são mais genéricos do que os padrões, descrevem um padrão recorrente (e de forma mais genérica).
- Outros autores utilizam padrão ou estilo no mesmo teor de descrição!

Padrões de arquitetura

- Um padrão arquitetural define a relação entre:
 - **Contexto:** Situação comum e recorrente que gera o problema analisado.
 - **Problema:** Descrição do problema e suas variações, forças opostas, restrições, etc. Geralmente os atributos de qualidade necessários são descrito aqui.
 - **Solução**

Padrões de arquitetura

- **Solução:** Arquitetura que resolve o problema, abstraído de forma adequada para ser generalizada. A solução descreve as responsabilidades e as relações entre os elementos
- A solução de um padrão é descrita usando:
 - Conjunto de elementos (repositórios de dados, processos e objetos)
 - Conjunto de mecanismos de interação e conectores (chamadas de métodos, eventos, canais de comunicação)
 - Organização topológica dos elementos (organização dos componentes)

Padrões de arquitetura

- A solução deve descrever de que forma os atributos de qualidade são providos pela configuração dos elementos
 - Contexto estático e dinâmico
- O conjunto {contexto, problema, solução} constitui o *template* para a documentação de um padrão

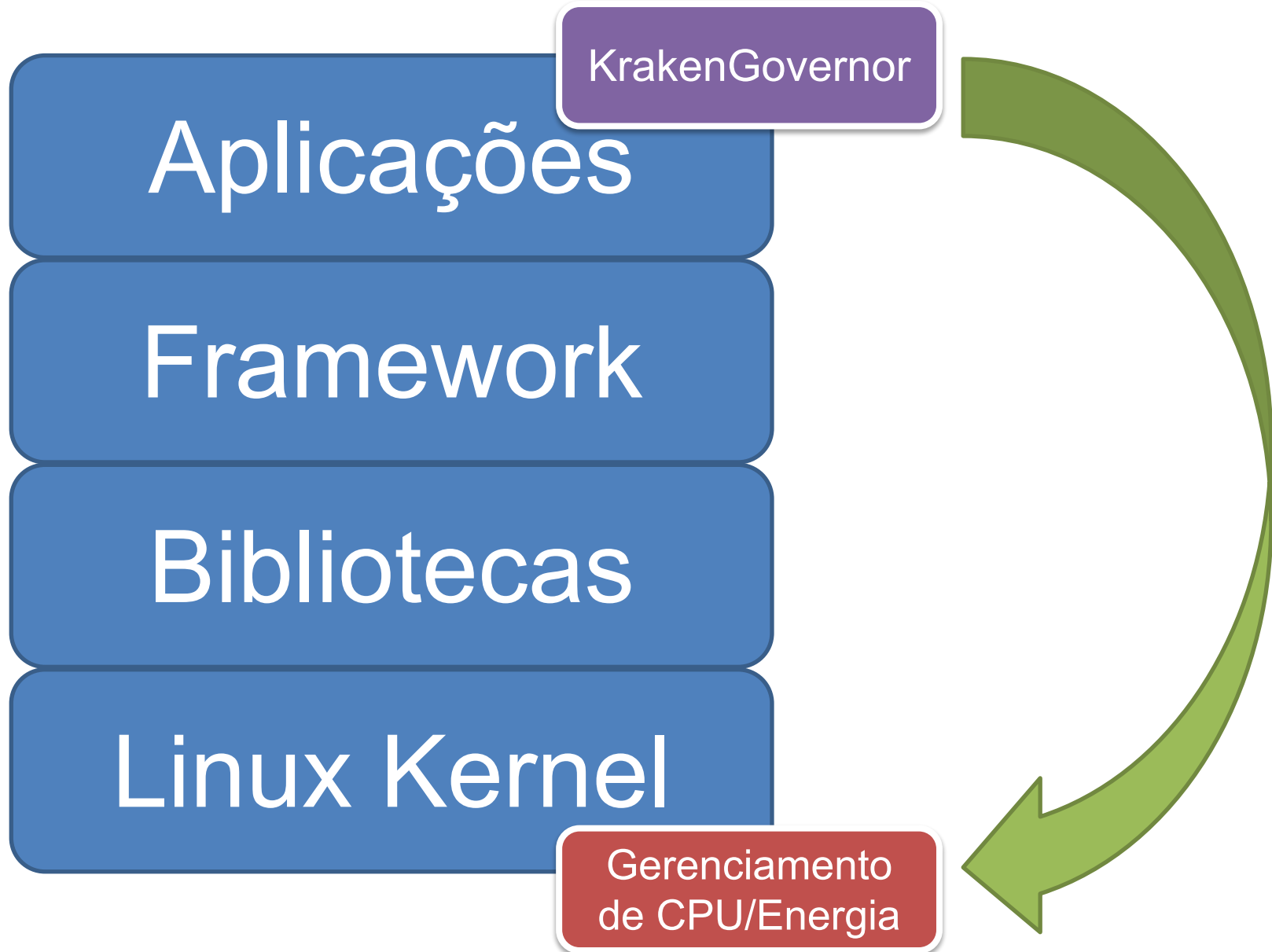
Padrões de arquitetura

- Sistemas complexos possuem diversos padrões em sua composição
- Exemplo:
 - Um sistema baseado na web pode fazer uso de um padrão arquitetural cliente-servidor de três camadas que usa:
 - Replicação de dados, firewalls, proxies, MVC e outros
 - Todos estes “micro padrões” também podem usar a estrutura de módulos em camadas

Padrões de arquitetura

- Para utilizar um padrão, devo aplicar exatamente tudo que ele descreve?
- Se eu fizer adaptações, não estarei mais usando o padrão?
- Quanto eu posso mudar para ainda seguir o padrão?

Posso fazer isso?



Posso fazer isso?

KrakenGovernor

Aplicações

Framework

Bibliotecas

Linux Kernel

Gerenciamento
de CPU/Energia

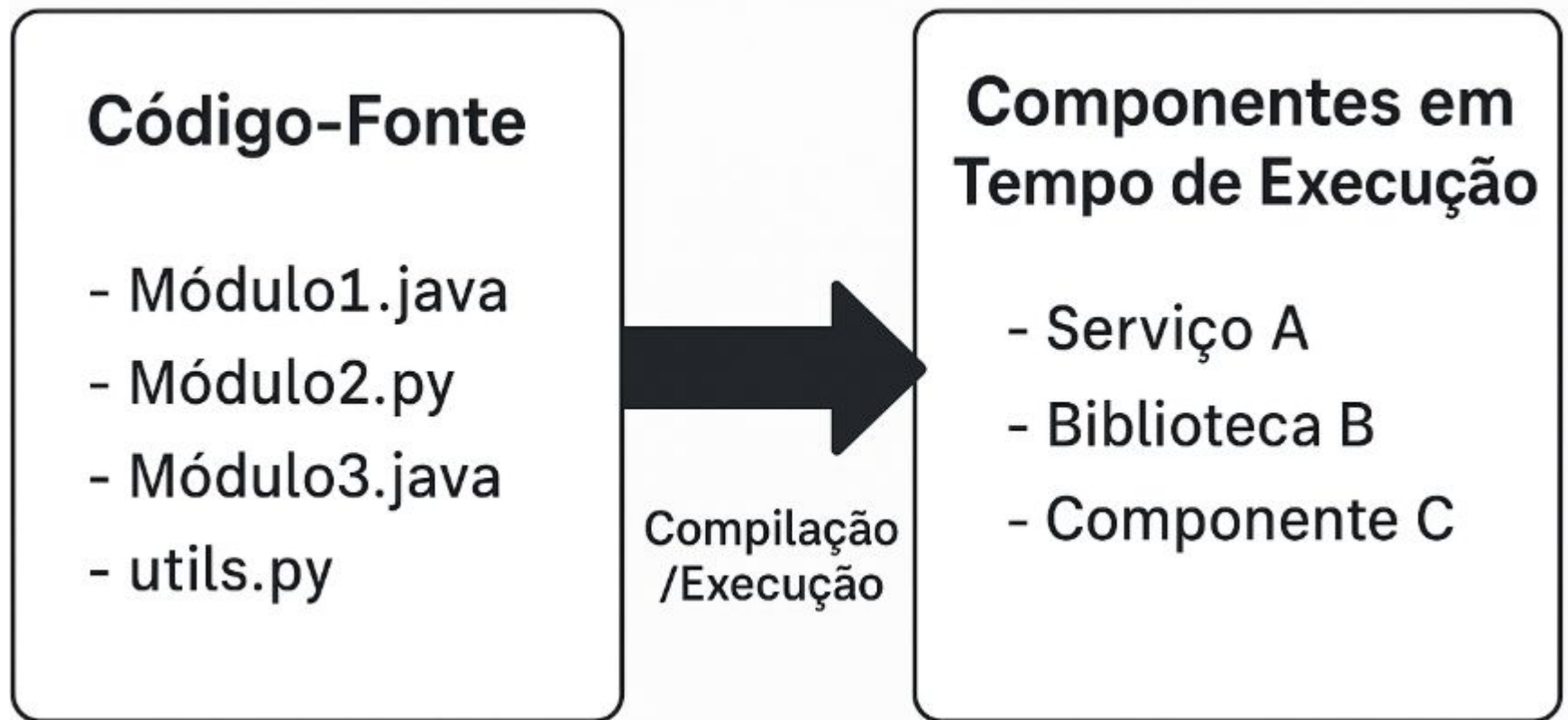
Quando a comunicação ocorre apenas com a camada imediatamente inferior, trata-se de uma **Arquitetura em Camadas Estrita (Strict Layered Architecture)**. Quando é permitido o "bypass" (ou pontes, como citado no slide anterior), chamamos de **Arquitetura em Camadas Relaxada (Relaxed Layered Architecture)**.

Em um padrão de camadas, a camada somente pode se comunicar com a mais próxima. Mesmo quebrando esta regra, ainda faço uso do padrão em camadas. Porém, ao fazer isso, estou ganhando desempenho e perdendo na separação de responsabilidades.

Padrões de arquitetura

- Os padrões podem ser categorizados pela quantidade de elementos que possuem:
 - Padrão modular (visão estática)
 - Módulos de software
 - Padrão Componente-e-Conector (C&C) (dinâmico)
 - Componentes e conectores entre estes elementos
 - Padrão de alocação
 - Módulos, componentes, conectores e elementos além de software
- . Geralmente o módulo é maior que o componente.
- . Um ou mais módulos do seu código-fonte, quando compilados e executados, dão origem a um ou mais componentes.

Padrões de arquitetura



Um componente pode ser formado por um ou mais módulos.

Padrões de arquitetura

- Alguns exemplos de padrões/estilos arquiteturais:
 - Divisão em Camadas (Layers)
 - Cliente-Servidor
 - Hexagonal
 - Orientada a Serviços / Microserviços
 - Model-View-Controller
 - Broker

Categoria 1: Estáticos

Padrões modulares: padrão de camadas

- **Contexto:** Sistemas complexos possuem a necessidade de evoluir porções do sistema de forma independente. Os desenvolvedores precisam de uma clara divisão e documentação da separação de interesses do sistema.
- **Problema:** O software precisa ser segmentado de forma que os módulos possam ser desenvolvimento e evoluídos de forma independente, com pouca interação entre as partes, permitindo portabilidade, modificabilidade e reuso.

Padrões modulares: padrão de camadas

- **Solução:** Para alcançar a separação de responsabilidades, a camada separa o software em unidades bem definidas, chamadas de camadas.
- Cada camada é um agrupamento de módulos que oferece um conjunto de serviços coesos (mesma categoria).
- Há restrições nas relações entre as camadas: devem ser unidirecionais.

Padrões modulares: padrão de camadas

- Cada camada possui uma interface para acesso.
- Há restrições no acesso em cada camada. Por exemplo, se temos a camada A e B, a implementação de A é permitida de acessar as funcionalidades providas pela camada B.
- Em alguns casos, módulos em uma camada podem precisar acessar, diretamente, módulos em outra camada que não é permitida a interação.

Padrões modulares: padrão de camadas

- Normalmente é permitida somente a interação com a camada mais abaixo. Fazer conexões que quebrem esta regra é chamado de ponte.
- Se muitas ocasiões de ponte acontecerem, o sistema pode não alcançar os objetivos desejados de portabilidade e modificabilidade.
- *Fazer uso de camadas acima não é permitido neste padrão.*

Padrões modulares: padrão de camadas

- É perceptível que vai haver inclusão de atrasos de comunicação pela adição de camadas, em troca de melhorar a modificabilidade e a portabilidade.
- **Adicionar muitas camadas pode aumentar a complexidade do sistema e dificultar o gerenciamento.**

Padrões modulares: padrão de camadas

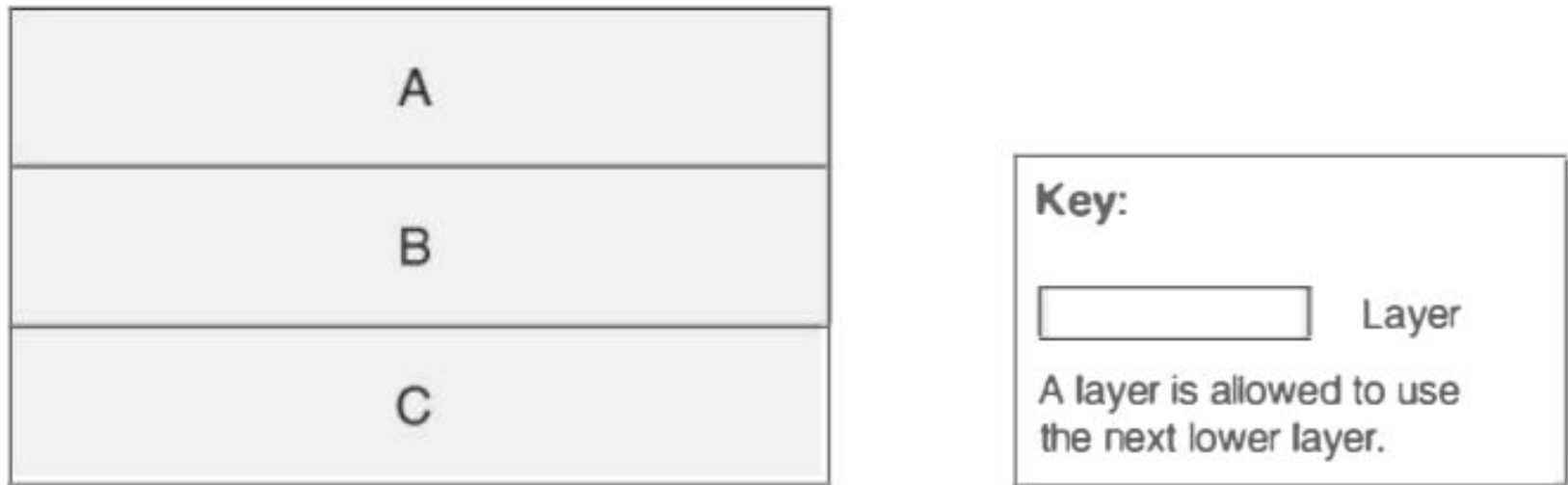
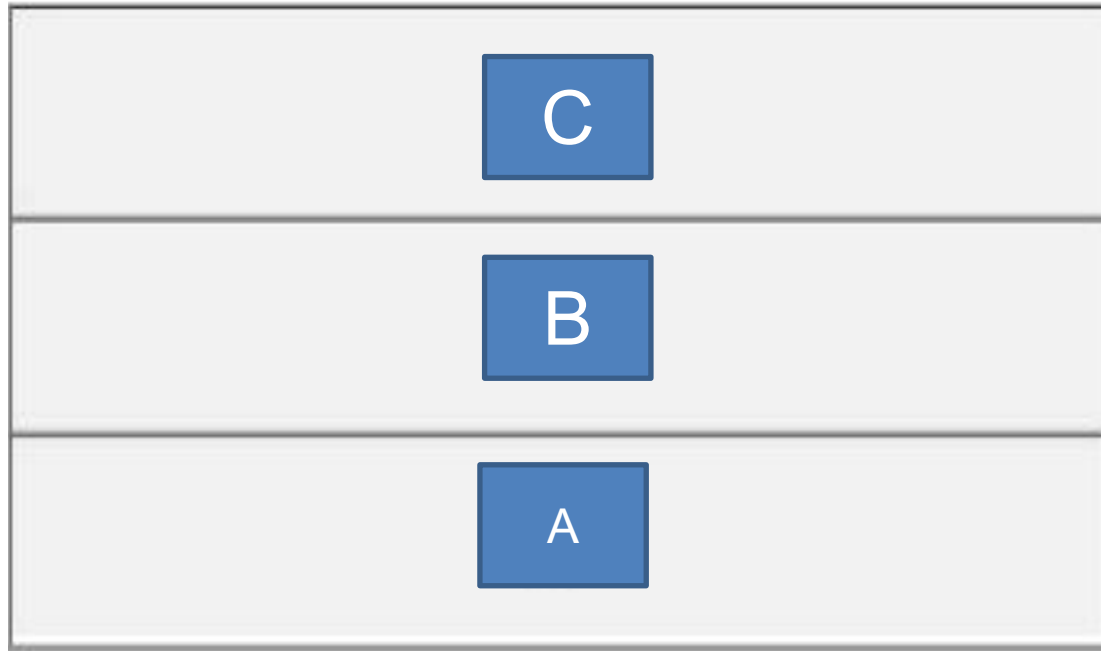


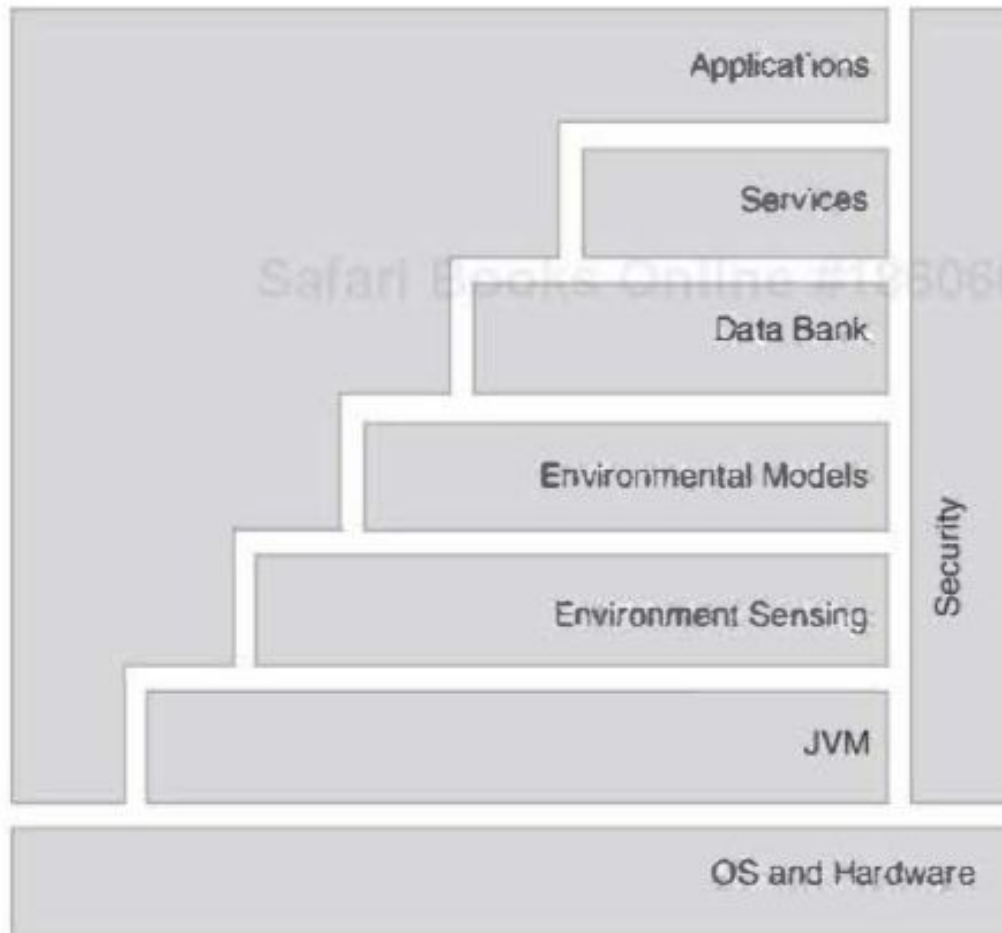
Figure 13.1. Stack-of-boxes notation for layered designs

Padrões modulares: padrão de camadas

- É a mesma coisa?



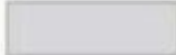
Padrões modulares: padrão de camadas



O que a camada Security representa?

Pq não colocar a camada Security logo acima da camada OS e Hardware já que todas acessam?

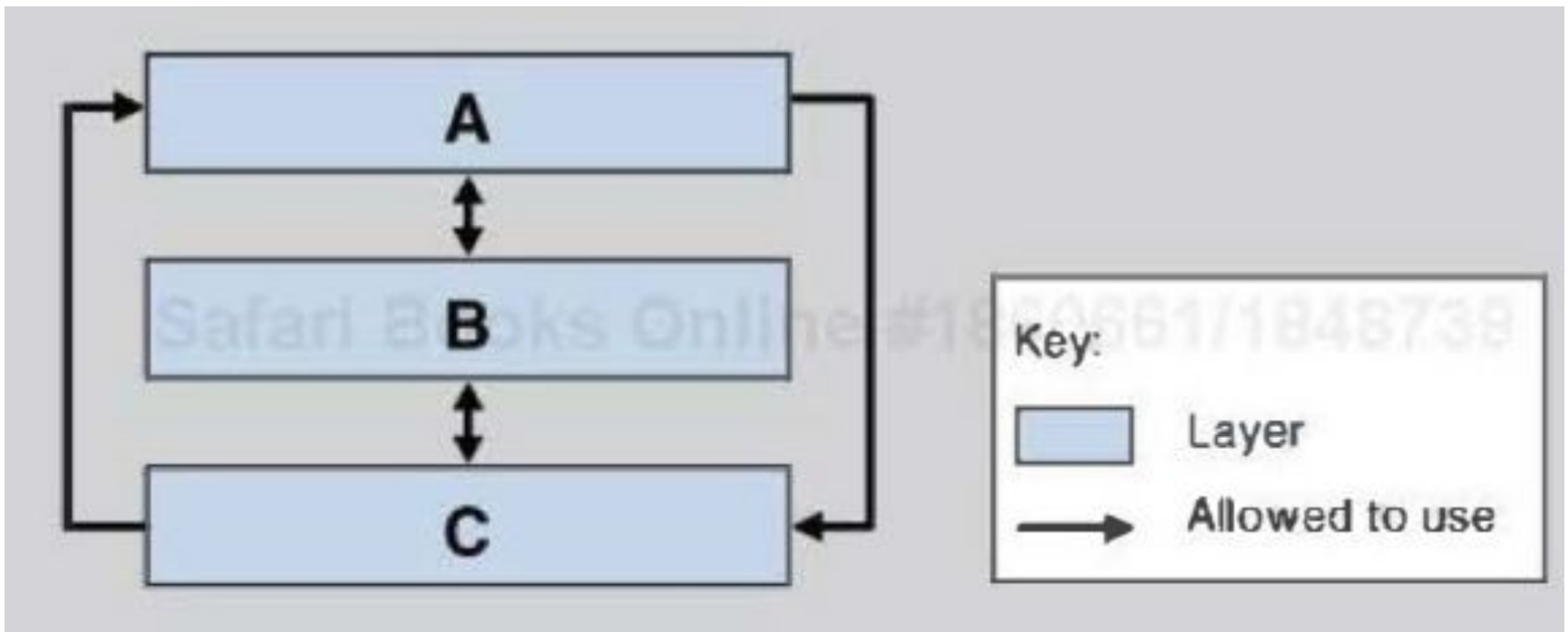
Key:

 layer

Software in a layer is allowed to use software in the same layer, or any layer immediately below or to the right.

Padrões modulares: padrão de camadas

- É uma arquitetura que usa o padrão em camadas de forma correta?

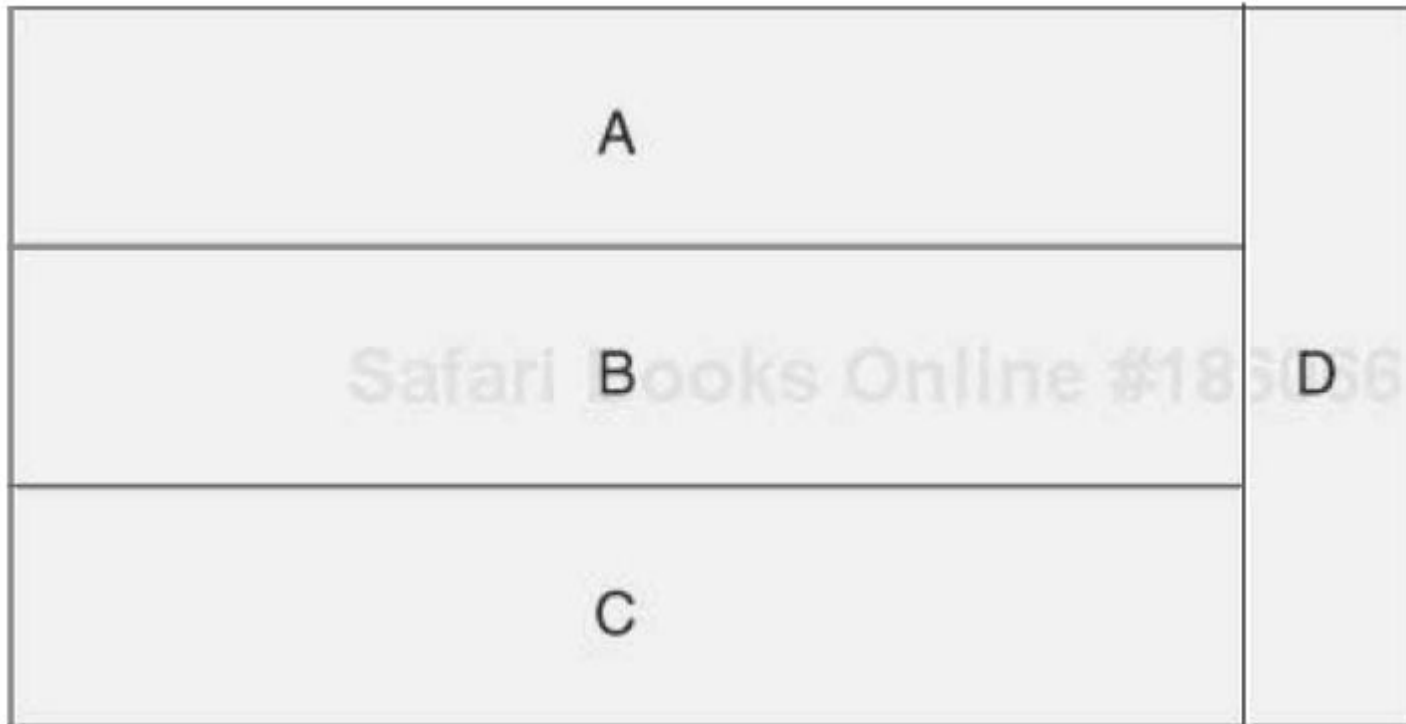


Padrões modulares: padrão de camadas

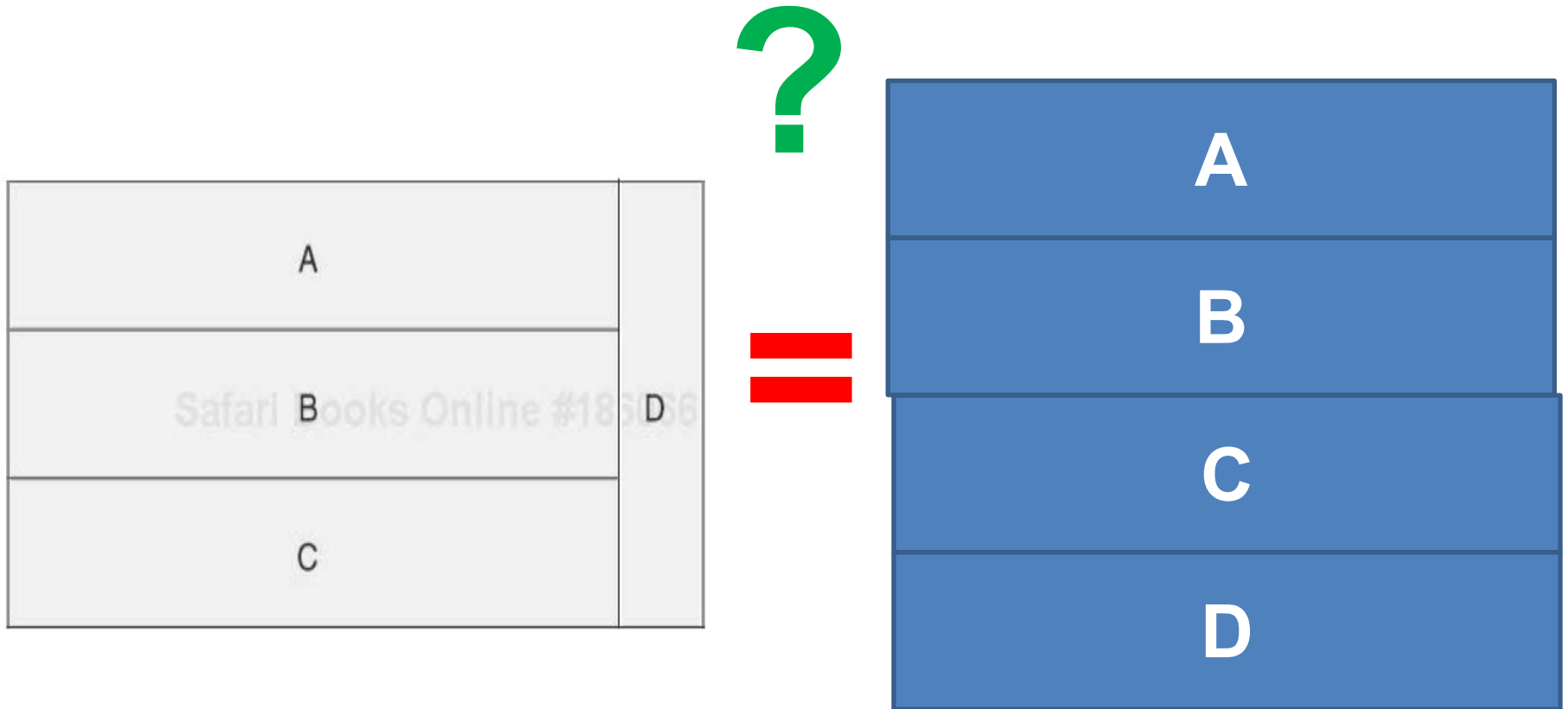
- Um conjunto de caixas ligadas não constitui o padrão em camadas.
- Uso das setas também não é permitido.
- Todo mundo se comunica com todo mundo, melhor desenhar de outra forma.
- Se a camada A é substituída por uma versão diferente, é necessário alterar B e C, pois fazem uso da camada A.
 - Reduz (ou remove?) o benefício do padrão em garantir modificabilidade e portabilidade

Padrões modulares: padrão de camadas

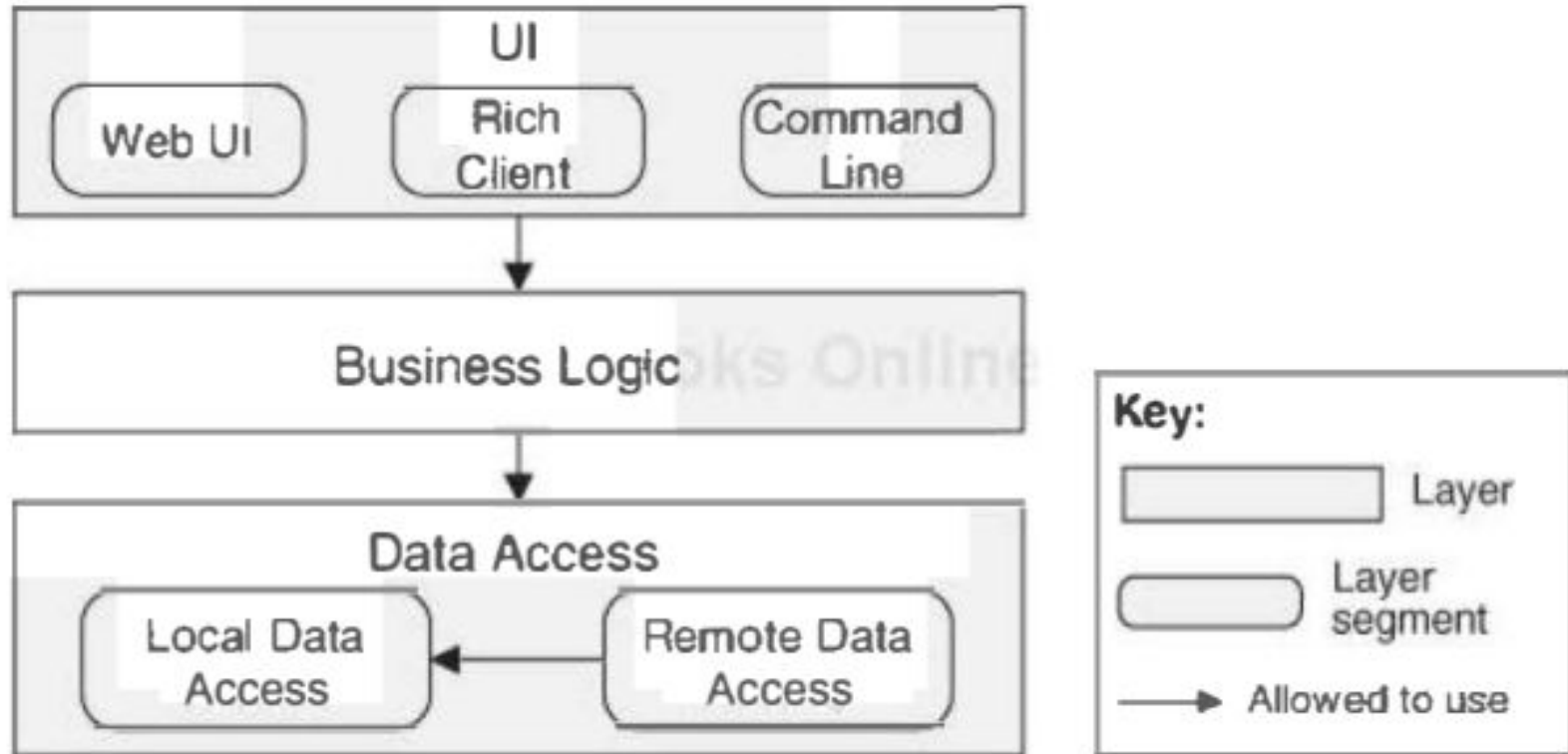
- E agora, está melhor? Está correto?



Padrões modulares: padrão de camadas



Padrões modulares: padrão de camadas



Padrões modulares: padrão de camadas

- A camada superior está usando as inferiores, mas há uma relação indicada pelas setas.
- Se remover as setas? Fica ilegível, não é possível identificar as interações.
- Recomendado é não usar este tipo de diagrama.
- Pelo diagrama em camadas, os módulos “Local Data Access” e “Remote Data Access” devem poder acessar uns aos outros de forma livre, pois estão na mesma camada.

Padrões modulares: padrão de camadas

- Um dos pontos mais importantes sobre o padrão em camadas é que uma camada não é permitida de acessar uma camada acima.
- Assumimos que um módulo (A) usa um módulo (B) quando (A) **DEPENDE** da resposta de (B).
- Porém, uma camada (B) é permitida de acessar uma camada acima (A) desde que (B) **não dependa** da resposta de (A).
 - Não gera alterações em (B) (dependência) se for preciso alterar (A).

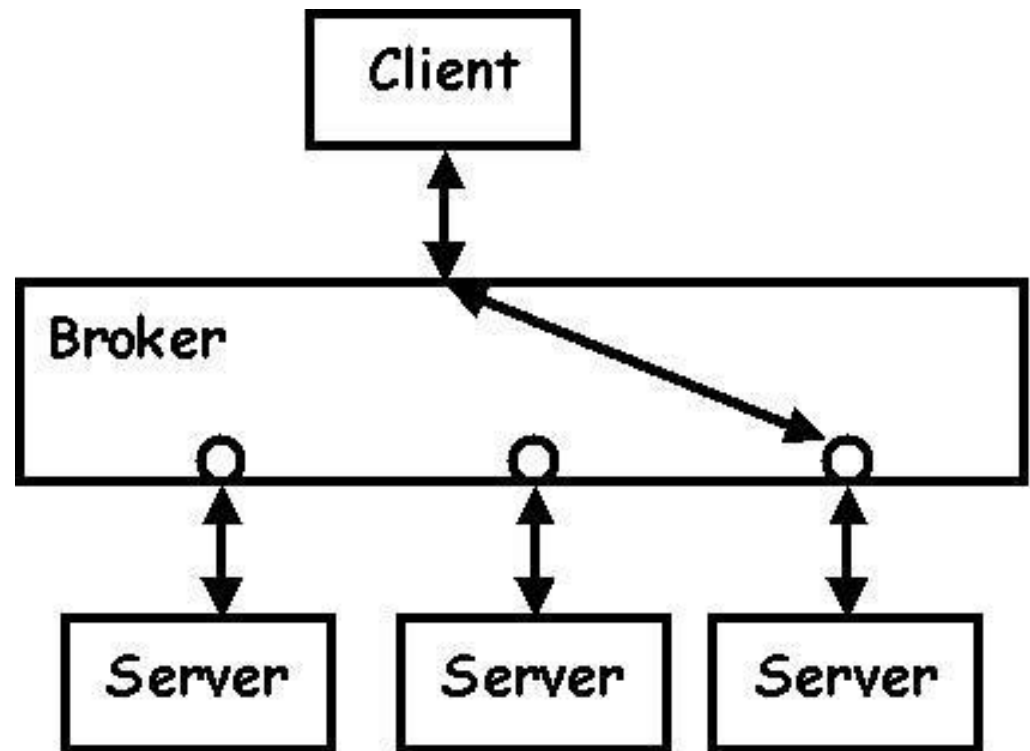
Padrões modulares: padrão de camadas

EXEMPLO

- A camada (A) chama uma função na camada (B) e envia um parâmetro.
- Este parâmetro é uma chamada a uma função de (A), por exemplo, acionar algum mecanismo de erro.
- Quando (B) invoca esta função em (A), ele não **depende** da resposta, é somente um acionador para outra coisa.
- A lógica fica toda em (A). Ao alterar (A), não preciso alterar (B).

Categoria 2: Dinâmicos

- Padrões C&C
 - Padrão broker



Padrões C&C: padrão broker

- **Contexto:** Alguns sistemas são construídos a partir de uma coleção de serviços distribuídos em servidores diversos.
- Implementar este tipo de sistema é complexo pois é preciso definir como os sistemas irão se comunicar e trocar dados.
- Também é necessário pensar sobre a disponibilidade dos servidores.
 - Se um deles cair? O que acontece com o restante do sistema?

Problema:

- Como estruturar os componentes de software de forma que os usuários não precisem saber sobre a natureza e localização dos serviços providos?
- Como tornar o sistema fácil de realizar mudanças de forma dinâmica alterando as conexões entre os usuários e os servidores?

Padrões C&C: padrão broker

- **Solução:** O padrão broker separa os usuários do serviço (clientes) dos provedores do serviço (servidores) pela inserção de uma entidade intermediária entre o cliente e o servidor, chamado de broker.
- Quando um cliente necessita de um serviço, o cliente requisita ao *broker* por uma interface de serviço.
- O broker encaminha a requisição do usuário ao servidor.

Padrões C&C: padrão broker

- O servidor processa a requisição do usuário.
- O resultado é encaminhado do servidor para o broker e então para o cliente.
- Desta forma, o cliente não conhece a identidade, localização ou características do servidor.
- Por conta dessa separação, se um servidor ficar indisponível, pode ser trocado de forma dinâmica, pelo *broker*.
- Se o servidor for substituído por um diferente, mas compatível, somente é preciso alterar o *broker*, sem afetar os clientes.

Padrões C&C: padrão broker

- Desvantagens do broker
 - Adiciona complexidade (adição do broker e protocolos de comunicação)
 - Aumento da latência por inserir um novo nível entre o cliente e o servidor
 - Depurar *brokers* pode ser difícil, pois executam em ambientes muito dinâmicos, onde é difícil replicar cenários de falha
 - O broker é um ponto óbvio de ataque (segurança)
 - O broker pode ser um ponto único de falha
 - *Brokers* podem ser gargalos de comunicação

Padrões C&C: padrão broker

Em sua essência, o padrão Broker tradicional centraliza a comunicação, onde um componente "cliente" envia uma requisição a um "broker" que, por sua vez, a encaminha para o serviço apropriado.

Isso promove o desacoplamento, pois o cliente não precisa conhecer a localização ou a implementação do serviço.

Um **API Gateway** pode ser visto como uma **evolução** direta e mais robusta do padrão *Broker*, atuando como um ponto único de entrada para todas as requisições de clientes a um conjunto de serviços de *backend*, tipicamente em uma arquitetura de microsserviços.

Exemplos de Tecnologias: Amazon API Gateway e NGINX.

Padrões C&C: padrão broker

Por outro lado, **Message Brokers** (ou sistemas de mensageria) são uma implementação moderna e poderosa do padrão *Broker* focada na comunicação **assíncrona**.

Em vez de uma requisição e resposta síncrona, os componentes trocam mensagens através de filas ou tópicos.

Exemplos de Tecnologias

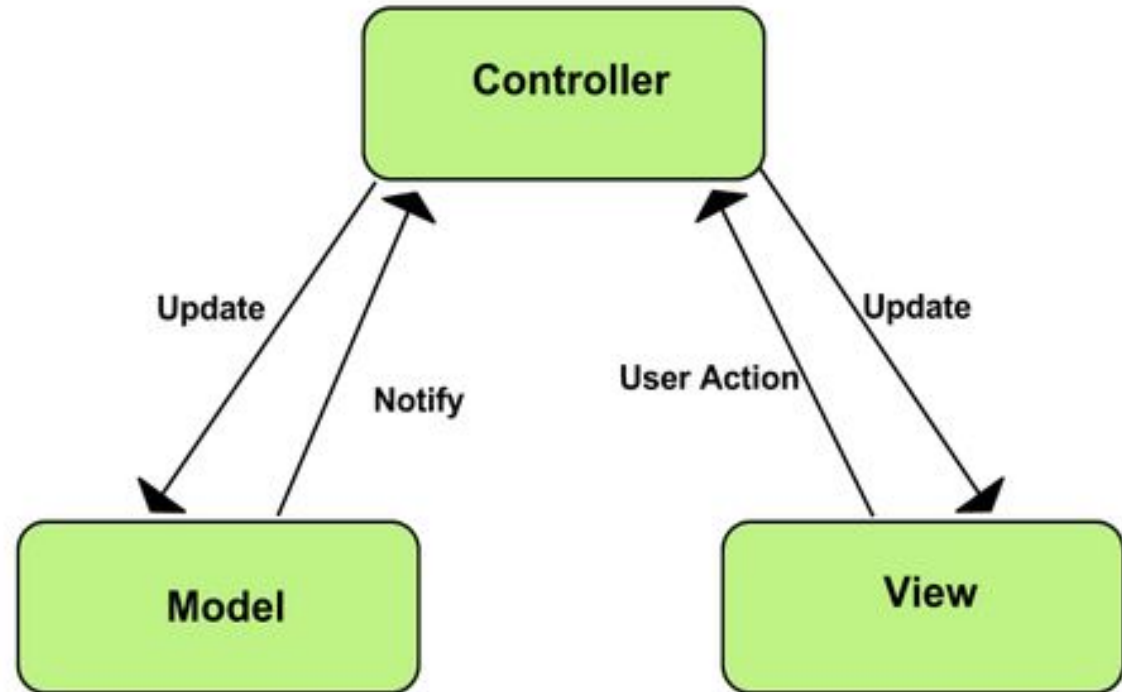
RabbitMQ, Apache Kafka, Amazon SQS, Google Cloud Pub/Sub.

Padrões C&C: padrão broker

- Percebem a diferença de padrões arquiteturais e tecnologias?
 - Diferença de arquitetura e tecnologia?
 - Broker vs Gateway vs Amazon API Gateway vs SQS

Padrões C&C: padrão MVC

- Padrões C&C
 - Model-View-Controller



Padrões C&C: padrão MVC

- **Contexto:** Geralmente, as interfaces de usuário são as entidades que mais sofrem mudanças em uma aplicação interativa.
- Por este motivo, é necessário manter a interação com o usuário em um repositório de código distinto do restante do sistema.

Padrões C&C: padrão MVC

- **Problema:** Como as funcionalidades do usuário podem ser separadas das funcionalidades da aplicação e mesmo assim manter o sistema responsivo as entradas do usuário.
- Como múltiplas visões do usuário podem ser criadas, mantidas e coordenadas quando os dados da aplicação mudam?

Padrões C&C: padrão MVC

- **Solução:** Separar as funcionalidades da aplicação em três categorias:
 - **Modelo:** deve conter os dados da aplicação
 - **Visão:** exibe partes dos dados e interage com o usuário
 - **Controlador:** deve mediar a comunicação entre o modelo e a visão e gerenciar notificações de mudanças de estado

Padrões C&C: padrão MVC

- MVC é apropriado para todas as situações?
 - Quando é indicado?

Padrões C&C: padrão MVC

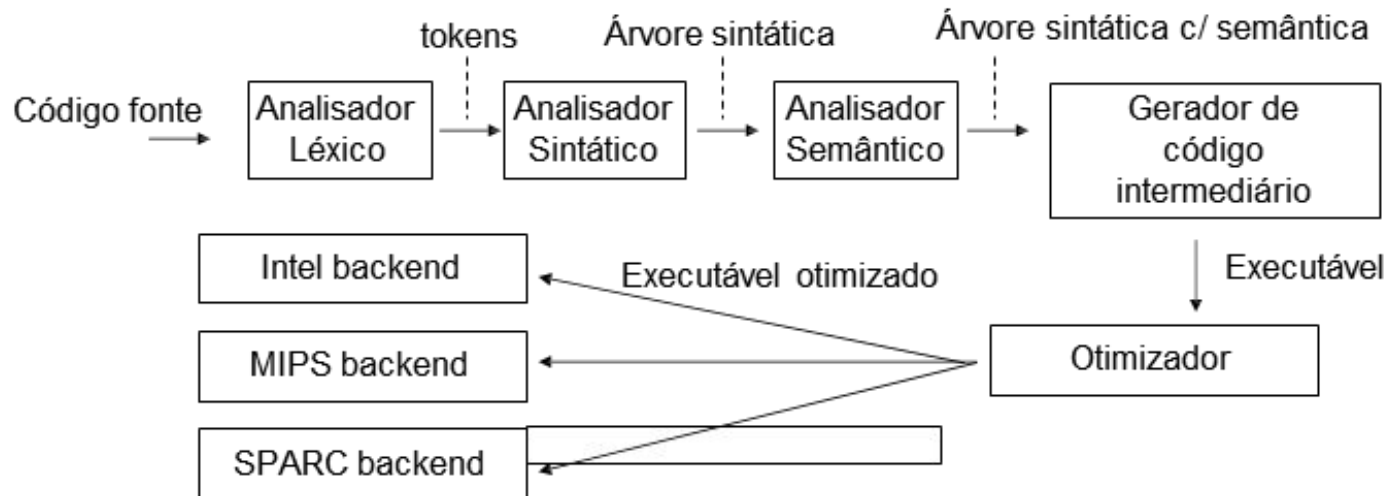
- Fazer uso de três camadas distintas e gerenciar várias formas de interação pode ser custoso e complexo.
- Para aplicações simples pode não fazer sentido adicionar esta complexidade.
- Como separar de forma precisa o que deve estar na Visão, Controlador e Modelo?
 - Complexo!
 - Tratamento/refinamento de dados deve ser feito no Modelo ou no Controlador?

Padrões C&C: padrão Pipes-and-Filters

- Padrões C&C
 - Pipes and Filters

Padrões C&C: padrão Pipes-and-Filters

- Problema
 - Desenvolvimento de um sistema que processa ou transforma um “stream” de dados.
- Exemplo:
 - Compilador



Padrões C&C: padrão Pipes-and-Filters

- Como dividir a tarefa entre vários desenvolvedores?
- E se for possível trocar etapas no processo?
- E recombinar etapas?

Padrões C&C: padrão Pipes-and-Filters

- Solução
 - Dividir a tarefa entre várias etapas sequenciais
 - Saída de uma etapa é a entrada da etapa seguinte
 - Cada etapa de processamento é implementada por um filtro (filter)
 - Consome e entrega os dados incrementalmente, ao invés de consumir todos os dados de uma só vez
 - Cada “pipe” implementa o fluxo dos dados entre os filtros

Padrões C&C: padrão Pipes-and-Filters

- Solução
 - Filter
 - Receber o dado da entrada
 - Processar o dado
 - Colocar o dado na saída
 - Pipe
 - Transferir o dado
 - Realizar “buffer”
 - Sincronizar os filtros “vizinhos”
 - Outros participantes
 - Fonte de dados
 - Saída de dados

Padrões C&C: padrão Pipes-and-Filters

- Consequências
 - Não é preciso criar arquivos intermediários (mas é possível)
 - Flexibilidade na troca de filtros
 - Flexibilidade na recombinação
 - Eficiência no processamento em paralelo
 - Vários filtros consumindo e produzindo dados em paralelo
- Ponto negativo
 - Gerenciamento de erros
 - Ausência de um estado global compartilhado
 - não sei como está o dado, só conheço na entrada e na saída do sistema
 - Filtro Gargalo
 - Como o processamento é sequencial ou em streams, se um filtro for significativamente mais lento que os outros, ele ditará a vazão (throughput) de todo o sistema, o que é um desafio clássico de escalabilidade nesse padrão.

Categoria 3: Alocação

Padrões Alocação: Três Camadas (Three-Tier Architecture)

Um dos exemplos mais claros de um Padrão de Implantação é a **Arquitetura de Três Camadas**.

Não confunda com o padrão lógico em camadas; aqui estamos falando da alocação física!

Neste padrão, os componentes de software são alocados em três conjuntos distintos de hardware (ou máquinas virtuais/contêineres)

Padrões Alocação: Três Camadas (Three-Tier Architecture)

Camada de Apresentação (Web Tier)

Componentes da interface do usuário são alocados em um ou mais servidores web (ex: NGINX, Apache).

Camada de Aplicação (Application Tier)

Componentes da lógica de negócio (onde o "trabalho pesado" acontece) são alocados em servidores de aplicação (ex: um servidor rodando Java com Spring Boot, Node.js, etc.).

Camada de Dados (Data Tier)

O sistema de gerenciamento de banco de dados e os dados em si são alocados em servidores de banco de dados.

Exemplo prático - Sistema de Vendas Online

1. Equipe de Front-End

Alocada na Camada de Apresentação

2. Equipe de Back-End / API

Alocada na Camada de Aplicação

3. Equipe de Dados / DBA

Alocada na Camada de Dados

Padrões Alocação: Três Camadas (Three-Tier Architecture)

1. Equipe de Front-End (Alocada na Camada de Apresentação)

Responsabilidade

Construir tudo que o usuário vê e interage no navegador.

Código-fonte que desenvolvem

O repositório ecommerce-ui.

Tecnologias

React, Angular, Vue.js, HTML, CSS.

O objetivo aqui é alocar servidores que permitem executar as tecnologias descritas.

Padrões Alocação: Três Camadas (Three-Tier Architecture)

1. Equipe de Front-End (Alocada na Camada de Apresentação)

Tarefas de desenvolvimento

Criar a página de listagem de produtos.

Desenvolver o componente do carrinho de compras.

Implementar os formulários de login e cadastro.

Como interagem

Eles não acessam o banco de dados.

O trabalho deles é consumir a API REST fornecida pela equipe de Back-end.

2. Equipe de Back-End / API

Responsabilidade

Implementar a lógica de negócio, as regras e a inteligência do sistema.

Código-fonte que desenvolvem

O repositório ecommerce-api.

Tecnologias

Java (Spring Boot), Python (Django), Node.js (Express).

Novamente, será preciso servidores próprios para alocação das tecnologias usadas.

2. Equipe de Back-End / API

Tarefas de desenvolvimento

Criar o endpoint GET /produtos para a equipe de Front-end listar os produtos.

Implementar a lógica para processar um pagamento, validando o estoque e o cartão de crédito.

Escrever o código que calcula o valor do frete.

Como interagem

Eles são a ponte.

Consomem os dados fornecidos pela equipe de Dados e expõem essa informação de forma segura e estruturada para a equipe de Front-end.

3. Equipe de Dados / DBA (Alocada na Camada de Dados)

Responsabilidade

Modelar, gerenciar e otimizar o banco de dados.

Garantir a integridade e o desempenho do acesso aos dados.

Código-fonte que desenvolvem

O repositório ecommerce-database-server.

Tecnologias

SQL, PostgreSQL, MySQL, Oracle.

Aqui, são necessários outros tipos de servidores (SOs com SGBDs).

Se pensar em escala, não é adequado colocar as 3 camadas executando na mesma máquina.

3. Equipe de Dados / DBA (Alocada na Camada de Dados)

Tarefas de desenvolvimento

Desenhar e criar as tabelas Produtos, Clientes e Pedidos.

Escrever scripts de migração para atualizar a estrutura do banco.

Otimizar consultas lentas que a equipe de Back-end reporta.

Como interagem

Não se preocupam com a aparência do site ou com as regras de negócio complexas.

O foco é puramente na estrutura, armazenamento e recuperação eficiente dos dados.

Dúvidas?



www.shutterstock.com · 744867163